

WDDA FS 2026: Leitfaden für Aufgabenserie 8

2026-05-20

1 Einleitung

Dieser Leitfaden bietet detaillierte Erklärungen für die Übungen in WDDA FS 2026 Aufgabenserie/Übungsblatt 8. Die Serie konzentriert sich auf **nichtlineare Zusammenhänge** (Variablentransformationen) und **kategoriale Prädiktoren** (Dummy-Variablen, Interaktionen). Wir lernen, wie das lineare Regressionsmodell erweitert werden kann, um auch nicht-lineare und nicht-numerische Zusammenhänge abzubilden.

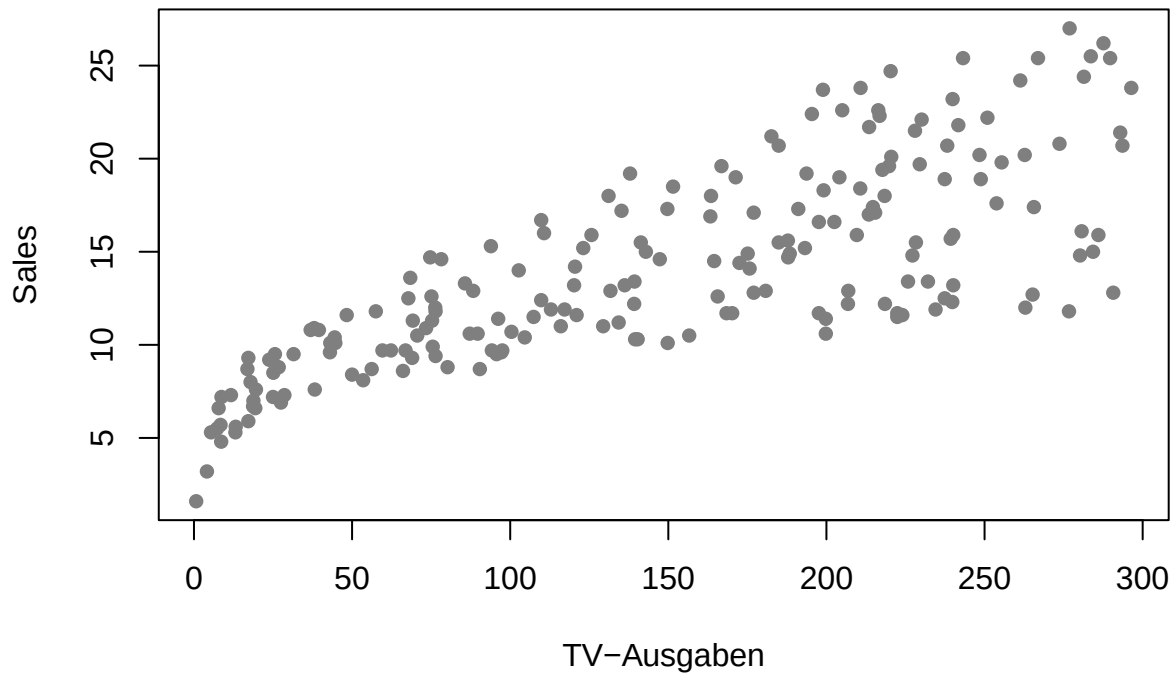
2 Aufgabe 1: Advertising – Nichtlineare Modelle für TV und Sales

Aufgabenstellung: Vergleichen Sie verschiedene Modellformen für den Zusammenhang zwischen TV-Ausgaben und Sales. Untersuchen Sie, ob eine Transformation die Modellgüte verbessert und ob die Annahmen der linearen Regression besser erfüllt werden.

2.1 Schritt 1: Daten einlesen und visualisieren (a)

```
adv <- read_excel("../data/WDDA_08.xlsx", sheet = "Advertising")
plot(adv$TV, adv$sales, pch = 16, col = "gray50",
     xlab = "TV-Ausgaben", ylab = "Sales",
     main = "Sales vs. TV")
```

Sales vs. TV



Bewertung: Die Daten zeigen einen klar positiven, aber gekrümmten Zusammenhang – der Effekt zusätzlicher TV-Ausgaben scheint bei hohen Werten abzuflachen (abnehmender Grenzertrag).

2.2 Schritt 2: Sechs Modelle anpassen (b)

```
attach(adv)

md1 <- lm(sales ~ TV)                                # Linear
md2 <- lm(sales ~ poly(TV, 2, raw = TRUE))           # Quadratisch
md3 <- lm(sales ~ sqrt(TV))                          # Wurzel
md4 <- lm(sales ~ log(TV))                          # Log(x)
md5 <- lm(log(sales) ~ log(TV))                     # Log-Log
md6 <- lm(log(sales) ~ TV)                          # Log(y)

cat("R2-Werte:\n")

## R2-Werte:
cat("Linear:      ", round(summary(md1)$r.squared, 4), "\n")

## Linear:      0.6119
cat("Quadratisch: ", round(summary(md2)$r.squared, 4), "\n")

## Quadratisch: 0.619
cat("Wurzel:      ", round(summary(md3)$r.squared, 4), "\n")

## Wurzel:      0.6229
cat("Log(x):      ", round(summary(md4)$r.squared, 4), "\n")
```

```
## Log(x):          0.565
cat("Log-Log:      ", round(summary(md5)$r.squared, 4), "\n")

## Log-Log:        0.7421
cat("Log(y):      ", round(summary(md6)$r.squared, 4), "\n")

## Log(y):        0.6156
```

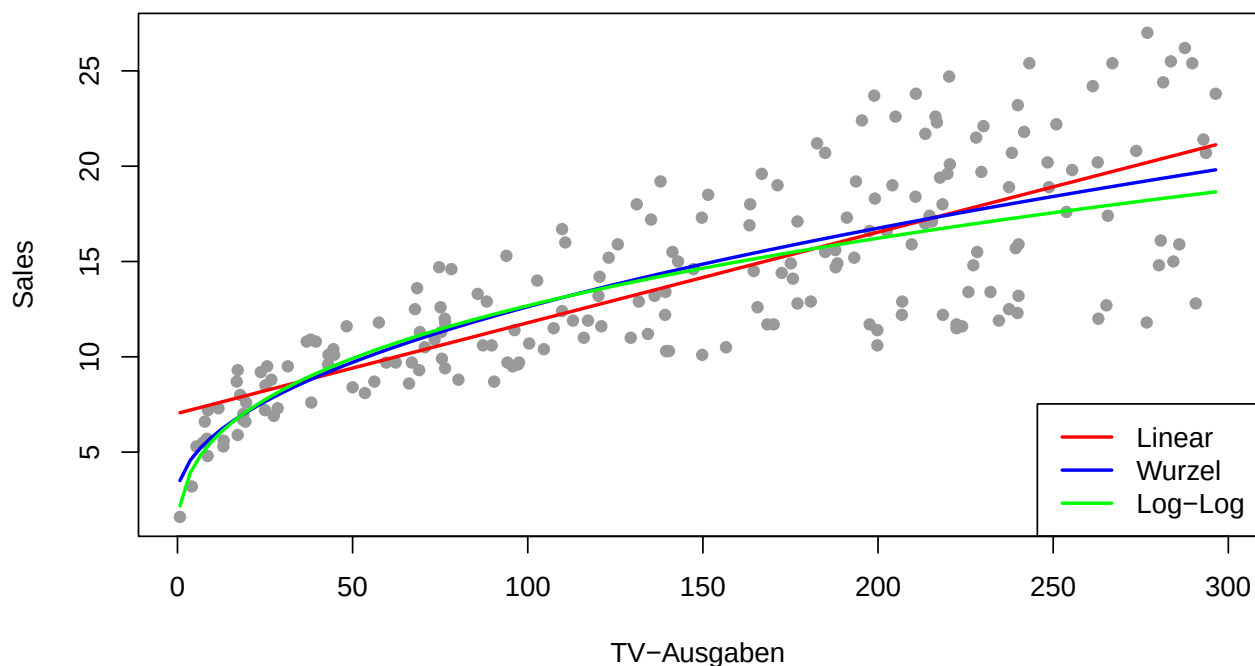
Wichtige Warnung: R^2 ist nur **direkt vergleichbar** zwischen Modellen mit identischer Response. Die Modelle md5 und md6 (mit $\log(y)$ als Response) erklären die Varianz von $\log(\text{sales})$, nicht von sales selbst – ihr R^2 ist also **nicht** direkt mit den anderen Modellen vergleichbar.

2.3 Schritt 3: Visualisierung der Modellkurven (c)

```
f1 <- function(x) coef(md1)[1] + coef(md1)[2] * x
f3 <- function(x) coef(md3)[1] + coef(md3)[2] * sqrt(x)
f5 <- function(x) exp(coef(md5)[1] + coef(md5)[2] * log(x))

plot(TV, sales, pch = 16, col = "gray60",
     xlab = "TV-Ausgaben", ylab = "Sales",
     main = "Vergleich der Regressionsmodelle")
curve(f1, add = TRUE, col = "red", lwd = 2)
curve(f3, add = TRUE, col = "blue", lwd = 2)
curve(f5, add = TRUE, col = "green", lwd = 2)
legend("bottomright", legend = c("Linear", "Wurzel", "Log-Log"),
     col = c("red", "blue", "green"), lwd = 2)
```

Vergleich der Regressionsmodelle

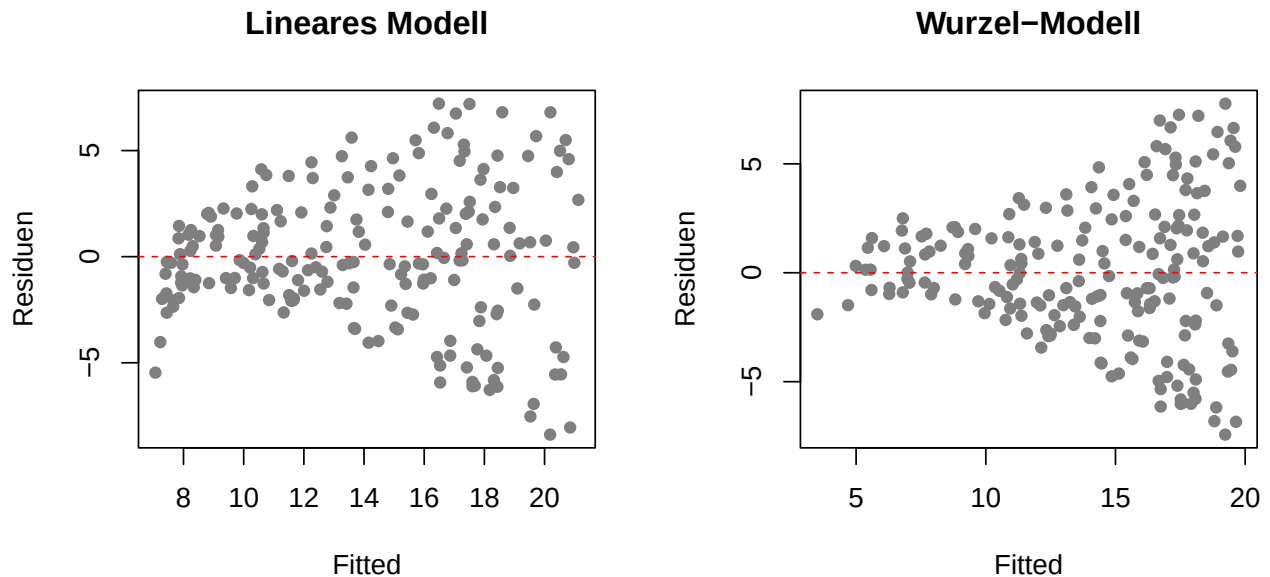


Bewertung: Wurzel- und Log-Log-Modell folgen der Datenwolke besser als das lineare Modell, insbesondere im rechten Bereich (hohe TV-Ausgaben).

2.4 Schritt 4: Residuenanalyse (d)

```
par(mfrow = c(1, 2))
plot(fitted(md1), resid(md1), pch = 16, col = "gray50",
     main = "Lineares Modell", xlab = "Fitted", ylab = "Residuen")
abline(h = 0, lty = 2, col = "red")

plot(fitted(md3), resid(md3), pch = 16, col = "gray50",
     main = "Wurzel-Modell", xlab = "Fitted", ylab = "Residuen")
abline(h = 0, lty = 2, col = "red")
```



```
par(mfrow = c(1, 1))
```

Bewertung:

- Beim linearen Modell zeigen die Residuen häufig ein **U-förmiges Muster** – ein Hinweis auf nichtlineare Strukturen, die das lineare Modell nicht erfasst.
- Beim Wurzel-Modell sind die Residuen meist gleichmässiger um Null gestreut.

2.5 Schritt 5: Interpretation des Log-Log-Modells (e)

```
summary(md5)
```

```
##
## Call:
## lm(formula = log(sales) ~ log(TV))
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.43349 -0.15917  0.01696  0.16910  0.39399
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   0.90525    0.07107   12.74  <2e-16 ***
## log(TV)        0.35504    0.01487   23.87  <2e-16 ***
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2109 on 198 degrees of freedom
## Multiple R-squared:  0.7421, Adjusted R-squared:  0.7408
## F-statistic: 569.8 on 1 and 198 DF,  p-value: < 2.2e-16
```

Interpretation:

- Die Steigung von $\log(\text{TV})$ ist eine **Elastizität**: eine Steigerung der TV-Ausgaben um 1% führt im Mittel zu einer Änderung der Sales um etwa $\beta_1\%$.
- Dies ist eine ökonomisch sehr intuitive Grösse, besonders für Marketing-Entscheidungen.

3 Aufgabe 2: Default – Kategoriale Prädiktoren

Aufgabenstellung: Untersuchen Sie für den Default-Datensatz, wie der Kontostand (`balance`) vom Studentenstatus (`student`) und Einkommen (`income`) abhängt. Verwenden Sie Dummy-Variablen und Interaktionen.

3.1 Schritt 1: Daten einlesen (a)

```
default <- read_excel("../data/WDDA_08.xlsx", sheet = "Default")
default$student <- factor(default$student, levels = c("No", "Yes"))
head(default)
```

```
## # A tibble: 6 x 4
##   default student balance income
##   <chr>    <fct>    <dbl> <dbl>
## 1 No      Yes      1056. 16001.
## 2 No      No       1136. 38475.
## 3 No      No       994. 26769.
## 4 No      No       862. 49672.
## 5 No      No       595. 43153.
## 6 No      No       758. 55279.
```

```
table(default$student)
```

```
##
## No Yes
## 708 292
```

3.2 Schritt 2: Einfaches Modell mit kategorialem Prädiktor (b)

```
mod_s <- lm(balance ~ student, data = default)
summary(mod_s)
```

```
##
## Call:
## lm(formula = balance ~ student, data = default)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -974.73 -379.64  -5.95  342.64 1444.62
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
```

```
## (Intercept)    776.35      18.14  42.794 < 2e-16 ***
## studentYes    198.38      33.57   5.909 4.72e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 482.7 on 998 degrees of freedom
## Multiple R-squared:  0.0338, Adjusted R-squared:  0.03284
## F-statistic: 34.92 on 1 and 998 DF,  p-value: 4.716e-09
```

Interpretation:

- **Intercept:** mittlerer Saldo für Nicht-Studenten (Baseline).
- **studentYes:** Differenz zwischen Studenten und Nicht-Studenten.
- Der p-Wert prüft, ob dieser Unterschied signifikant ist.

3.3 Schritt 3: Manuelle Dummy-Kodierung (c)

```
default$st10 <- ifelse(default$student == "Yes", 1, 0)
mod_s_dummy <- lm(balance ~ st10, data = default)
```

```
cat("Koeffizienten Faktor-Modell:\n")
```

```
## Koeffizienten Faktor-Modell:
```

```
print(coef(mod_s))
```

```
## (Intercept)  studentYes
##    776.3500    198.3827
```

```
cat("\nKoeffizienten Dummy-Modell:\n")
```

```
##
```

```
## Koeffizienten Dummy-Modell:
```

```
print(coef(mod_s_dummy))
```

```
## (Intercept)      st10
##    776.3500    198.3827
```

Bewertung: Beide Modelle sind **identisch**. Die manuelle Dummy-Variable ist nur eine alternative Schreibweise.

3.4 Schritt 4: Multiple Regression mit Income (d)

```
mod_si <- lm(balance ~ income + student, data = default)
summary(mod_si)
```

```
##
```

```
## Call:
```

```
## lm(formula = balance ~ income + student, data = default)
```

```
##
```

```
## Residuals:
```

```
##      Min       1Q   Median       3Q      Max
## -977.73 -380.66   -3.71   345.16 1444.88
```

```
##
```

```
## Coefficients:
```

```
##              Estimate Std. Error t value Pr(>|t|)
```

```
## (Intercept)  7.918e+02  7.158e+01  11.062  < 2e-16 ***
## income      -3.857e-04  1.729e-03  -0.223  0.823513
## studentYes  1.899e+02  5.075e+01   3.741  0.000193 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 482.9 on 997 degrees of freedom
## Multiple R-squared:  0.03385,    Adjusted R-squared:  0.03191
## F-statistic: 17.47 on 2 and 997 DF,  p-value: 3.5e-08
```

Interpretation:

- **income:** erwartete Änderung in balance pro Einheit Einkommenszuwachs, **bei konstantem Studentenstatus.**
- **studentYes:** Unterschied im Saldo zwischen Studenten und Nicht-Studenten, **bei konstantem Einkommen.**
- Geometrisch: zwei **parallele** Regressionsgeraden – nur das Niveau (Intercept) unterscheidet sich.

3.5 Schritt 5: Modell mit Interaktion (e)

```
mod_si_int <- lm(balance ~ income * student, data = default)
summary(mod_si_int)

##
## Call:
## lm(formula = balance ~ income * student, data = default)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1007.17  -383.56    -3.95   343.47  1444.69
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    7.809e+02  7.397e+01  10.558  <2e-16 ***
## income        -1.139e-04  1.790e-03  -0.064   0.9493
## studentYes     2.692e+02  1.443e+02   1.865   0.0624 .
## income:studentYes -4.063e-03  6.922e-03  -0.587   0.5573
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 483.1 on 996 degrees of freedom
## Multiple R-squared:  0.03419,    Adjusted R-squared:  0.03128
## F-statistic: 11.75 on 3 and 996 DF,  p-value: 1.435e-07
```

Interpretation:

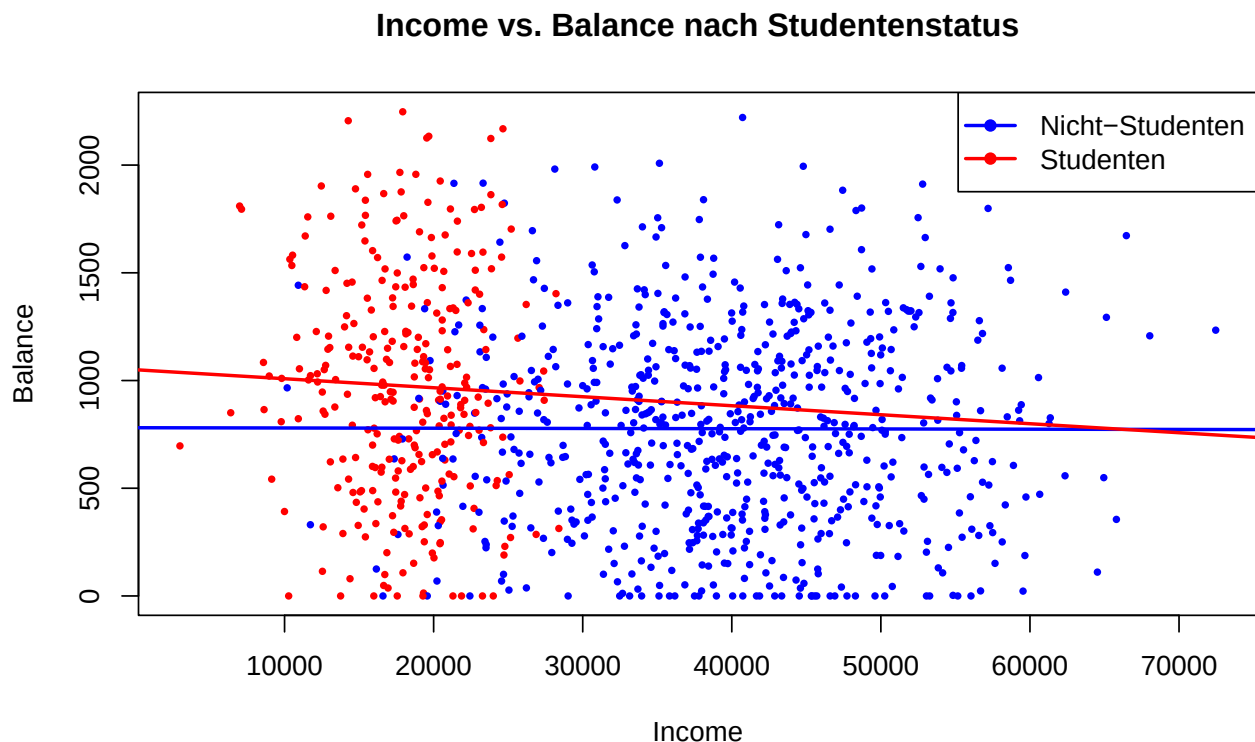
- **income:** Einkommenseffekt für Nicht-Studenten (Baseline).
- **studentYes:** Differenz im Intercept zwischen den Gruppen.
- **income:studentYes:** Differenz im **Steigungskoeffizienten** – wie stark unterscheidet sich der Einkommenseffekt zwischen Studenten und Nicht-Studenten?
- Eine **signifikante Interaktion** bedeutet: der Effekt eines Prädiktors hängt vom Wert des anderen ab; die Regressionsgeraden sind **nicht mehr parallel**.

3.6 Schritt 6: Visualisierung der Interaktion (f)

```
plot(default$income, default$balance,
     col = ifelse(default$student == "Yes", "red", "blue"),
     pch = 16, cex = 0.6,
     xlab = "Income", ylab = "Balance",
     main = "Income vs. Balance nach Studentenstatus")

abline(a = coef(mod_si_int)[1],
       b = coef(mod_si_int)[2],
       col = "blue", lwd = 2)
abline(a = coef(mod_si_int)[1] + coef(mod_si_int)[3],
       b = coef(mod_si_int)[2] + coef(mod_si_int)[4],
       col = "red", lwd = 2)

legend("topright", legend = c("Nicht-Studenten", "Studenten"),
      col = c("blue", "red"), pch = 16, lwd = 2)
```



Bewertung: Sind die beiden Geraden ungefähr parallel? Dann ist die Interaktion gering. Unterscheiden sich die Steigungen deutlich, spricht das für eine echte Interaktion.

3.7 Schritt 7: Modellvergleich (g)

```
anova(mod_si, mod_si_int)

## Analysis of Variance Table
##
## Model 1: balance ~ income + student
## Model 2: balance ~ income * student
##   Res.Df      RSS Df Sum of Sq    F Pr(>F)
## 1      997 232536475
```



```
## 2      996 232456061 1      80414 0.3445 0.5573
```

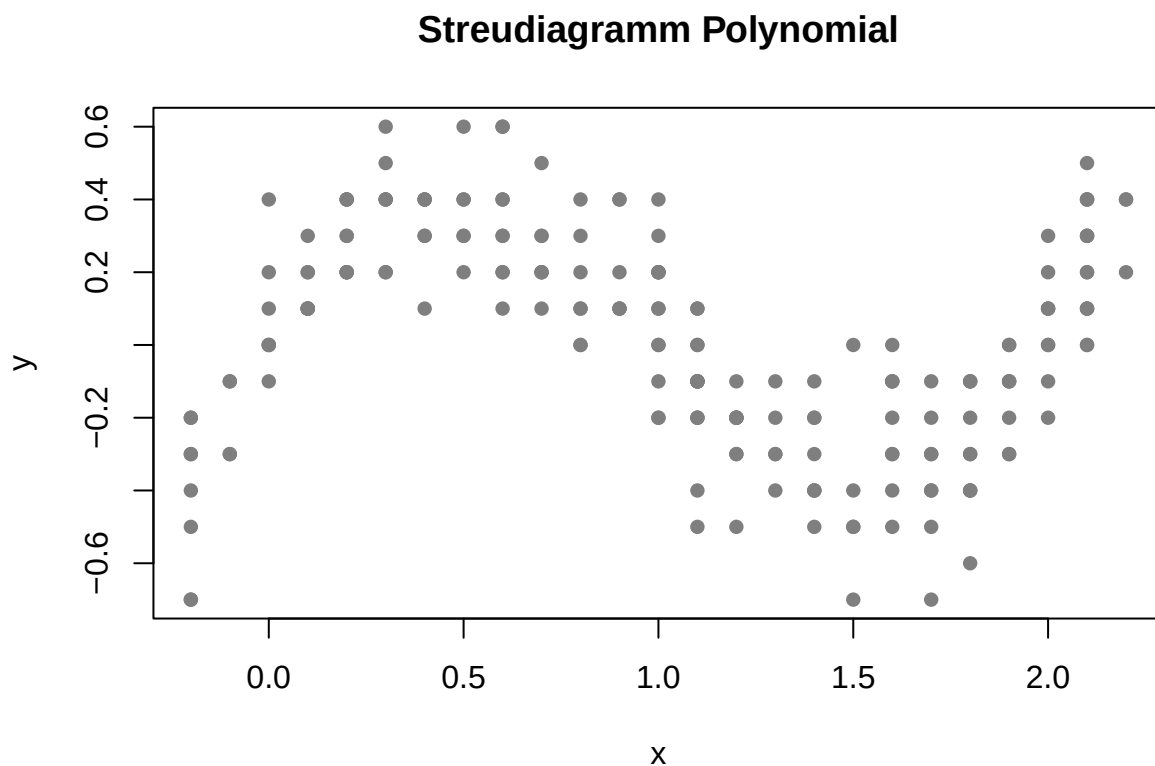
Interpretation: Der F-Test prüft, ob das Modell mit Interaktion signifikant besser ist als das Modell ohne Interaktion. Falls $p > 0.05$: der Zusatznutzen der Interaktion ist statistisch nicht belegbar – das einfachere Modell wird bevorzugt.

4 Aufgabe 3: Polynomial – Polynomianpassung

Aufgabenstellung: Untersuchen Sie für den Polynomial-Datensatz, welcher Polynomgrad den Zusammenhang zwischen x und y am besten beschreibt.

4.1 Schritt 1: Daten einlesen und visualisieren (a)

```
poly_dat <- read_excel("../data/WDDA_08.xlsx", sheet = "Polynomial")
plot(poly_dat$x, poly_dat$y, pch = 16, col = "gray50",
     xlab = "x", ylab = "y",
     main = "Streudiagramm Polynomial")
```



Bewertung: Klar nichtlinearer Verlauf – ein lineares Modell wird die Daten nicht gut beschreiben.

4.2 Schritt 2: Polynome verschiedener Grade anpassen (b)

```
m1 <- lm(y ~ x, data = poly_dat) # Linear
m2 <- lm(y ~ poly(x, 2, raw = TRUE), data = poly_dat) # Quadratisch
m3 <- lm(y ~ poly(x, 3, raw = TRUE), data = poly_dat) # Kubisch
m4 <- lm(y ~ poly(x, 4, raw = TRUE), data = poly_dat) # Grad 4

cat("R2-Werte:\n")
```

```
## R2-Werte:
```

```
cat("Grad 1: ", round(summary(m1)$r.squared, 4), "\n")
```

```
## Grad 1: 0.0525
```

```
cat("Grad 2: ", round(summary(m2)$r.squared, 4), "\n")
```

```
## Grad 2: 0.0526
```

```
cat("Grad 3: ", round(summary(m3)$r.squared, 4), "\n")
```

```
## Grad 3: 0.7286
```

```
cat("Grad 4: ", round(summary(m4)$r.squared, 4), "\n")
```

```
## Grad 4: 0.7286
```

4.3 Schritt 3: Modellvergleich mit F-Tests (c)

```
anova(m1, m2, m3, m4)
```

```
## Analysis of Variance Table
```

```
##
```

```
## Model 1: y ~ x
```

```
## Model 2: y ~ poly(x, 2, raw = TRUE)
```

```
## Model 3: y ~ poly(x, 3, raw = TRUE)
```

```
## Model 4: y ~ poly(x, 4, raw = TRUE)
```

```
##   Res.Df    RSS Df Sum of Sq    F Pr(>F)
## 1     198 16.6745      0      0.0000    NA      NA
## 2     197 16.6732      1    0.0013  0.0521 0.8198
## 3     196  4.7762      1   11.8970 485.7965 <2e-16 ***
## 4     195  4.7755      1    0.0008  0.0307 0.8612
```

```
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

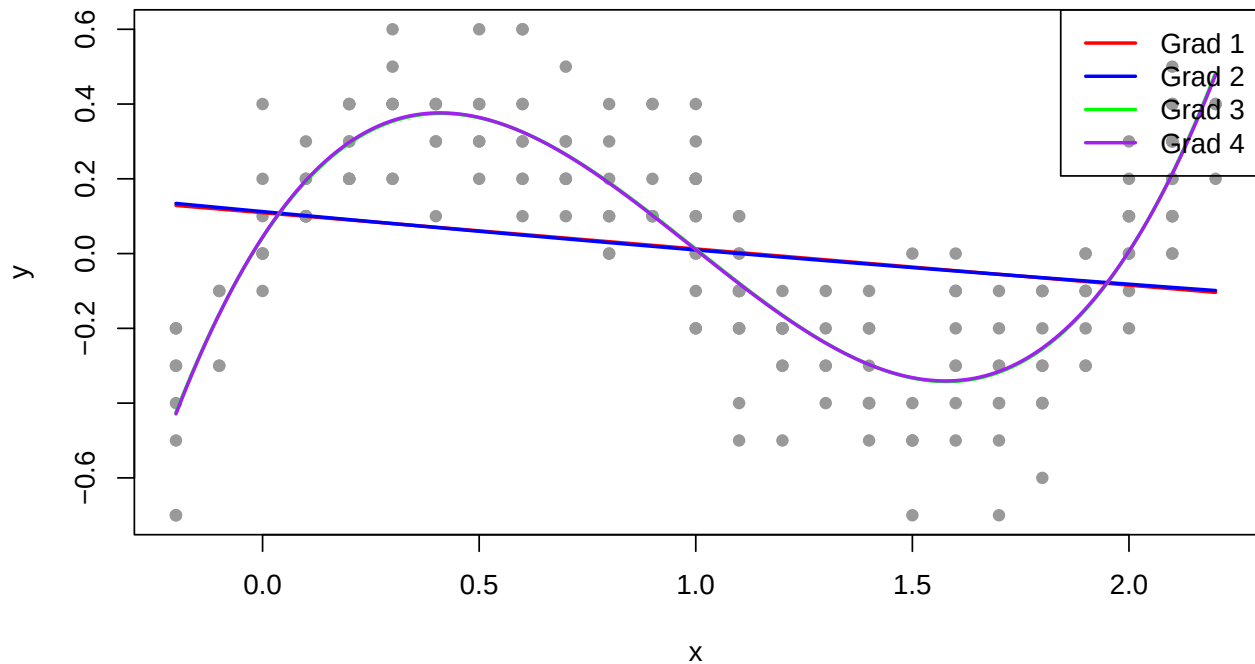
Interpretation: Die ANOVA-Tabelle zeigt, ob jeweils der nächsthöhere Polynomgrad eine signifikante Verbesserung bringt. Ab dem Polynomgrad, bei dem $p > 0.05$, lohnt sich die zusätzliche Komplexität nicht mehr.

4.4 Schritt 4: Visualisierung der angepassten Kurven (d)

```
x_grid <- seq(min(poly_dat$x), max(poly_dat$x), length.out = 200)
```

```
plot(poly_dat$x, poly_dat$y, pch = 16, col = "gray60",
     xlab = "x", ylab = "y",
     main = "Polynomanpassungen verschiedener Grade")
lines(x_grid, predict(m1, newdata = data.frame(x = x_grid)),
     col = "red", lwd = 2)
lines(x_grid, predict(m2, newdata = data.frame(x = x_grid)),
     col = "blue", lwd = 2)
lines(x_grid, predict(m3, newdata = data.frame(x = x_grid)),
     col = "green", lwd = 2)
lines(x_grid, predict(m4, newdata = data.frame(x = x_grid)),
     col = "purple", lwd = 2)
legend("topright", legend = c("Grad 1", "Grad 2", "Grad 3", "Grad 4"),
     col = c("red", "blue", "green", "purple"), lwd = 2)
```

Polynomanpassungen verschiedener Grade



Bewertung: Höhere Grade folgen den Daten enger, aber Vorsicht: zu hohe Polynomgrade führen zu **Overfitting** – das Modell passt sich an zufällige Schwankungen an, ohne den wahren Zusammenhang besser zu beschreiben.

5 Zusammenfassung

Diese Aufgabenserie hat gezeigt:

1. **Variablentransformationen** (Wurzel, Logarithmus, Polynome) erweitern das lineare Modell auf nichtlineare Zusammenhänge.
2. **R^2 -Vergleiche** sind nur sinnvoll, wenn die abhängige Variable dieselbe ist (Vorsicht bei Log-Transformation der Response).
3. **Residuenanalysen** sind unverzichtbar, um zu beurteilen, ob ein Modell die Daten gut beschreibt – Muster in den Residuen verraten Modellfehler.
4. **Kategoriale Prädiktoren** werden über Dummy-Variablen kodiert. Die Wahl der Baseline beeinflusst die Interpretation der Koeffizienten, aber nicht die Vorhersagen.
5. **Interaktionen** erlauben es, dass der Effekt eines Prädiktors vom Wert eines anderen abhängt – geometrisch entspricht das nicht-parallelen Regressionsgeraden.
6. **F-Tests** zum Modellvergleich helfen, den richtigen Komplexitätsgrad zu wählen – einfacher ist besser, solange das einfachere Modell nicht signifikant schlechter passt.